

PHÂN LOẠI CIFAR-10 BẰNG DEEP LEARNING VÀ VISION TRANSFORMER

Hồ Võ Hoàng Duy - 25C01002

Ngày 3 tháng 10 năm 2026

Tóm tắt

Báo cáo nghiên cứu phân loại ảnh CIFAR-10 bằng các mô hình học sâu AlexNet, VGG11, ResNet18 và Vision Transformer (ViT tiny). Các kiến trúc được điều chỉnh cho ảnh 32×32 , cài đặt bằng các lớp cơ bản của PyTorch và huấn luyện từ trọng số ngẫu nhiên, không sử dụng pretrained. Thực nghiệm dùng 45.000 ảnh huấn luyện, 5.000 ảnh kiểm định và 10.000 ảnh kiểm tra; mỗi mạng được huấn luyện 200 epoch và chọn checkpoint theo accuracy kiểm định. Sáu cấu hình Pixel, HOG và SIFT phẳng kết hợp SVM hoặc Random Forest được giữ làm mốc so sánh. AlexNet đạt accuracy 83,87% và macro-F1 0,8381, cao nhất trong các cấu hình đã chạy; ViT tiny đạt 68,21%. Báo cáo phân tích hiệu quả học biểu diễn, chi phí huấn luyện và giới hạn tổng quát hóa của các mô hình.

1 Giới thiệu

Phân loại ảnh yêu cầu mô hình nhận biết đối tượng dù góc nhìn, màu sắc và nền thay đổi. CIFAR-10 gồm 60.000 ảnh màu 32×32 thuộc 10 lớp, là bộ dữ liệu phù hợp để khảo sát bài toán này ở độ phân giải thấp [7]. Thay vì cố định cách trích xuất đặc trưng trước khi học bộ phân lớp, học sâu đồng thời tối ưu biểu diễn ảnh và quyết định phân lớp từ dữ liệu.

Báo cáo tập trung vào hai hướng: mạng tích chập (CNN) và Vision Transformer. AlexNet dùng chuỗi tích chập, ReLU và pooling; VGG tổ chức các tích chập nhỏ theo chiều sâu; ResNet bổ sung kết nối tắt để học phần dư [8, 10, 5]. ViT biểu diễn ảnh bằng một chuỗi patch và dùng self-attention để trao đổi thông tin giữa các vùng ảnh [4].

Mục tiêu là cài đặt các biến thể AlexNet, VGG11, ResNet18 và ViT tiny cho CIFAR-10, huấn luyện từ đầu và đánh giá accuracy cùng macro-F1 trên tập kiểm tra chính thức. Các kết quả truyền thống được giữ lại để đo mức cải thiện so với biểu diễn pixel thô hoặc đặc trưng thiết kế thủ công. Phần phương pháp tập trung vào học sâu; các baseline chỉ được mô tả ngắn gọn để giải thích phép so sánh.

2 Dữ liệu và tiền xử lý

CIFAR-10 cung cấp 50.000 ảnh train và 10.000 ảnh test chính thức [7]. Từ tập train, phép chia phân tầng với seed 42 giữ 45.000 ảnh để huấn luyện và 5.000 ảnh để kiểm định. Test giữ nguyên 10.000 ảnh và được đánh giá sau khi chọn checkpoint của mỗi mạng.

Bảng 1: Phân chia dữ liệu cho các thí nghiệm.

Tập	Số ảnh	Số ảnh/lớp	Vai trò
Huấn luyện	45.000	4.500	Học thống kê chuẩn hóa và trọng số
Kiểm định	5.000	500	Chọn checkpoint học sâu
Kiểm tra	10.000	1.000	Đánh giá cuối

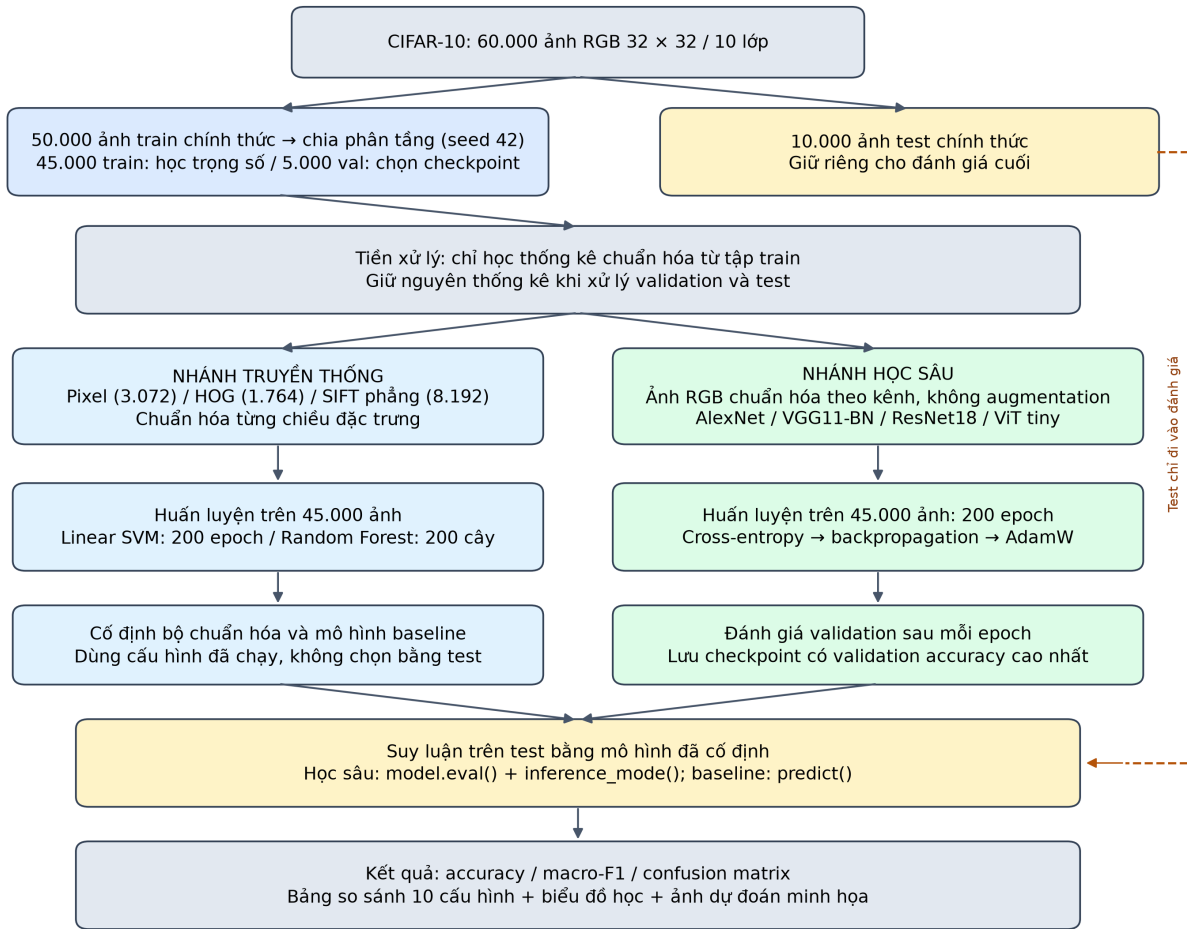
Đầu vào học sâu. Ảnh giữ nguyên ba kênh RGB và kích thước 32×32 . Giá trị pixel được chuyển về $[0, 1]$, rồi chuẩn hóa theo từng kênh:

$$\hat{I}_{c,h,w} = \frac{I_{c,h,w}/255 - \mu_c^{\text{train}}}{\max(\sigma_c^{\text{train}}, 10^{-6})}. \quad (1)$$

Trung bình và độ lệch chuẩn được tính trên tập train, sau đó giữ cố định cho validation và test. Lần chạy này không dùng tăng cường dữ liệu như lật ảnh, cắt ngẫu nhiên hoặc biến đổi màu.

Đầu vào baseline. Pixel thô được trải phẳng thành 3.072 chiều. HOG và SIFT dùng ảnh xám $I = 0.299R + 0.587G + 0.114B$; vector đặc trưng được chuẩn hóa theo từng chiều bằng thống kê của train. Hai nhánh dùng cách chuẩn hóa phù hợp với loại đầu vào, không dùng thống kê test để học tham số.

2.1 Tổng quan pipeline



Hình 1: Pipeline tổng quan: hai nhánh dùng cùng cỡ dữ liệu, chỉ học thống kê chuẩn hóa từ train; validation chọn checkpoint học sâu và test chỉ đi vào đánh giá cuối.

Hình 1 phân biệt bước học tham số với bước đánh giá. Nhánh truyền thống dùng đặc trưng cố định rồi học SVM hoặc Random Forest; nhánh học sâu tối ưu đồng thời biểu diễn và bộ phân lớp. Sau khi cố định mô hình và thống kê chuẩn hóa, test được xử lý và suy luận để tạo các chỉ số so sánh. Ảnh dự đoán minh họa được lấy từ checkpoint đã chọn, không dùng để chọn lại trọng số hoặc điều chỉnh siêu tham số.

3 Phương pháp học sâu

Các kiến trúc trong phần này được mô tả theo cài đặt tại `deep_models/models.py`. Kích thước tensor được ghi theo thứ tự $C \times H \times W$ cho CNN; $T \times D$ cho ViT; bỏ qua chiều batch B . Đây là các biến thể cho CIFAR-10, với trọng số được học từ đầu.

3.1 Nguyên lý chung: học biểu diễn và tối ưu đầu-cuối

Tích chập. Một lớp convolution tạo mỗi kênh đầu ra bằng cách kết hợp các vùng lân cận trên tất cả kênh đầu vào:

$$a_{o,h,w} = b_o + \sum_{c=1}^{C_{in}} \sum_{u,v} W_{o,c,u,v} x_{c,sh+u-p,sw+v-p}. \quad (2)$$

Kernel W được dùng lại ở nhiều vị trí; s là stride, p là padding. Với kích thước đầu vào H , kernel k và dilation 1, kích thước đầu ra là $H_{out} = \lfloor (H + 2p - k)/s \rfloor + 1$. Ví dụ, kernel 3×3 , padding 1 và stride 1 giữ nguyên kích thước; stride 2 giảm khoảng một nửa. Việc chia sẻ trọng số giúp cùng một mẫu cạnh được phát hiện ở nhiều vị trí mà không học một bộ trọng số riêng cho từng pixel.

Phi tuyến và gộp thông tin. ReLU, $\max(0, a)$, làm mạng có khả năng biểu diễn quan hệ phi tuyến. Nếu chỉ ghép các lớp tuyến tính, toàn mạng vẫn là một phép biến đổi tuyến tính, dù có nhiều lớp. Max-pooling 2×2 , stride 2 chọn đáp ứng lớn

nhất trong mỗi vùng, giảm kích thước và độ nhảy với dịch chuyển nhỏ; nó cũng có thể làm mất chi tiết trên ảnh nhỏ. Global average pooling lấy trung bình trên toàn bộ vùng không gian của từng kênh. Qua nhiều lớp, vùng ảnh ảnh hưởng đến một đặc trưng (receptive field) mở rộng; biểu diễn có thể kết hợp mẫu cục bộ thành cấu trúc lớn hơn phục vụ phân lớp.

Học biểu diễn. Mạng phân loại được viết thành $z = g_\theta(f_\phi(I))$, trong đó f_ϕ là bộ trích xuất đặc trưng học được, g_θ là bộ phân lớp và z có 10 logit. Với đặc trưng thu công $h(I)$, chỉ bộ phân lớp được học; với học sâu, cả ϕ và θ nhận gradient từ cùng mục tiêu. Chẳng hạn, nếu một mẫu màu hoặc đường biên giúp phân biệt *ship* với *airplane*, trọng số phía trước có thể được điều chỉnh để làm nổi bật mẫu đó. Đây là khả năng của mô hình, không phải khẳng định rằng mọi bộ lọc trong lần chạy này đã được giải thích về mặt ngữ nghĩa.

Hàm mất mát và quy trình cập nhật. Với logit z_{ik} :

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{10} \exp(z_{ij})}, \quad \mathcal{L}_{CE} = -\frac{1}{B} \sum_{i=1}^B \log p_{i, y_i}. \quad (3)$$

Mỗi mini-batch được chuẩn hóa, đưa qua mạng, tính cross-entropy, lan truyền ngược và cập nhật bằng AdamW. Đạo hàm loss theo logit của một mẫu tỉ lệ với $p_{ik} - \mathbf{1}[k = y_i]$, nên lớp sai có điểm cao nhận tín hiệu điều chỉnh mạnh. Gradient norm được giới hạn ở 1,0 trước bước optimizer; weight decay 10^{-4} giúp kiểm soát độ lớn trọng số. Các cơ chế này không bảo đảm hết quá khớp, vì còn phụ thuộc dữ liệu, kiến trúc và thời gian huấn luyện.

Đánh giá. Sau mỗi epoch, mạng chuyển sang chế độ evaluation để đo accuracy validation và lưu checkpoint tốt nhất. Khi suy luận, dropout được vô hiệu hóa, BatchNorm dùng thống kê đã tích lũy; trọng số không cập nhật. Nhấn cuối là $\hat{y} = \arg \max_k z_k$. Softmax chỉ chuyển logit thành phân bố điểm tổng bằng 1; điểm cao không bảo đảm dự đoán đúng hay xác suất đã được hiệu chuẩn.

3.2 AlexNet với GroupNorm

Ý tưởng kiến trúc. AlexNet sử dụng tích chập, ReLU, pooling và bộ phân lớp có dropout [8]. Biến thể CIFAR-10 gồm năm convolution, với GroupNorm sau mỗi lớp. Các convolution có stride 1; padding 2 cho kernel 5×5 và padding 1 cho kernel 3×3 . Vì vậy, chỉ các lớp pooling làm giảm kích thước không gian.

Bảng 2: Luồng tensor của AlexNet trong thí nghiệm. Conv được theo sau bởi GroupNorm và ReLU.

Bước	Thao tác	Đầu ra
Đầu vào	RGB đã chuẩn hóa	$3 \times 32 \times 32$
1	Conv 5×5 , 64 kênh; max-pool	$64 \times 16 \times 16$
2	Conv 3×3 , 192 kênh; max-pool	$192 \times 8 \times 8$
3	Conv 3×3 , 384 kênh	$384 \times 8 \times 8$
4	Conv 3×3 , 256 kênh	$256 \times 8 \times 8$
5	Conv 3×3 , 256 kênh; max-pool	$256 \times 4 \times 4$
6	Adaptive average pooling 2×2 ; flatten	1024
7	Dropout; Linear $1024 \rightarrow 512$; ReLU	512
8	Dropout; Linear $512 \rightarrow 10$	10 logit

GroupNorm. Tại mỗi mẫu, các kênh được chia thành tám nhóm. Trung bình và phương sai được tính trên các kênh và vị trí không gian trong từng nhóm, không trên batch [11]. Nếu $\mathcal{G}$ là nhóm chứa a_j :

$$\hat{a}_j = \gamma_j \frac{a_j - \mu_{\mathcal{G}}}{\sqrt{\sigma_{\mathcal{G}}^2 + \epsilon}} + \beta_j. \quad (4)$$

Với lớp 64 kênh, mỗi nhóm có tám kênh; với lớp 192 kênh, mỗi nhóm có 24 kênh. Hệ số scale và bias được học cùng mạng. GroupNorm không cần running mean và running variance như BatchNorm, nên cách tính thống kê không phụ thuộc số mẫu trong mini-batch. Không thể từ đó kết luận GroupNorm là nguyên nhân duy nhất khiến AlexNet đạt kết quả cao nhất; cần thí nghiệm thay riêng lớp chuẩn hóa.

Bộ phân lớp và điều chuẩn. Adaptive pooling giữ lưới 2×2 , cho phép vector cuối còn thông tin vị trí thô. Hai lớp tuyến tính học tổ hợp phi tuyến của các đặc trưng tích chập. Dropout 0,5 ngẫu nhiên loại bỏ một nửa phần tử ở vị trí dropout khi train và điều chỉnh tỉ lệ phần tử còn lại; khi inference dùng toàn bộ phần tử. Cơ chế này hạn chế việc bộ phân lớp phụ thuộc quá mạnh vào một số đặc trưng. Mạng có 2.786.890 tham số. So với AlexNet ImageNet, ảnh đầu vào, kernel, pooling, chuẩn hóa và classifier đều đã được điều chỉnh; vì vậy tên kiến trúc cần được đọc cùng cấu hình trên.

3.3 VGG11-BN với số kênh giảm

Ý tưởng kiến trúc. VGG dùng các convolution 3×3 xếp chồng [10]. Bản cài đặt dùng tám convolution, mỗi lớp có padding 1, stride 1, theo sau bởi BatchNorm và ReLU. Hai lớp 3×3 liên tiếp có vùng nhìn danh nghĩa 5×5 trong trường hợp stride 1, đồng thời bổ sung một phép phi tuyến giữa chúng. Năm pooling giảm dần kích thước ảnh về 1×1 .

Bảng 3: Năm stage của VGG11-BN. Mỗi stage kết thúc bằng max-pooling 2×2 .

Stage	Các convolution 3×3	Đầu ra sau pooling	Số conv
1	$3 \rightarrow 32$	$32 \times 16 \times 16$	1
2	$32 \rightarrow 64$	$64 \times 8 \times 8$	1
3	$64 \rightarrow 128 \rightarrow 128$	$128 \times 4 \times 4$	2
4	$128 \rightarrow 256 \rightarrow 256$	$256 \times 2 \times 2$	2
5	$256 \rightarrow 256 \rightarrow 256$	$256 \times 1 \times 1$	2

BatchNorm. Với mỗi kênh, BatchNorm chuẩn hóa activation bằng trung bình và phương sai của các mẫu và vị trí không gian trong mini-batch lúc train, rồi học scale và bias [6]. Khi evaluation, lớp này dùng thống kê tích lũy thay vì tính lại theo batch test. Điều này làm thao tác train và inference khác nhau; chế độ `model.eval()` là cần thiết để suy luận đúng. Khác với GroupNorm của AlexNet, thống kê lúc train phụ thuộc batch.

Phân lớp và đánh đổi. Sau stage 5, flatten tạo 256 đặc trưng, đưa thẳng vào lớp tuyến tính $256 \rightarrow 10$; không dùng các fully connected lớn của VGG nguyên bản và không có dropout trong cài đặt này. Mạng có 2.310.186 tham số, ít nhất trong bốn mạng. Cấu trúc tuần tự dễ triển khai nhưng việc giảm $32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ có thể làm mất chi tiết ở ảnh nhỏ. Dữ liệu không xác nhận riêng tác động của pooling; đây là một yếu tố cần kiểm tra khi giải thích chênh lệch với các mạng khác.

3.4 ResNet18 với kết nối phần dư

BasicBlock. Mỗi block chứa hai convolution 3×3 kèm BatchNorm; ReLU đặt sau convolution thứ nhất và sau phép cộng nhánh tắt. Viết chi tiết:

$$F(x) = \text{BN}_2(\text{Conv}_2(\text{ReLU}(\text{BN}_1(\text{Conv}_1(x))))), \quad (5)$$

$$y = \text{ReLU}(F(x) + S(x)). \quad (6)$$

Khi số kênh và kích thước không đổi, $S(x) = x$. Khi đổi kênh hoặc stride 2, S là convolution 1×1 với cùng stride và BatchNorm, để hai nhánh có cùng shape trước phép cộng. Hai convolution trong nhánh F không có bias vì đã đi kèm BatchNorm.

Vì sao học phần dư? Thay vì học toàn bộ ánh xạ $H(x)$, nhánh biến đổi học $F(x) = H(x) - x$ khi dùng identity shortcut [5]. Bỏ qua ReLU để xét phép cộng, đạo hàm theo đầu vào là $\partial(F(x) + x)/\partial x = \partial F/\partial x + I$. Hạng I tạo đường truyền trực tiếp cho gradient. ReLU sau phép cộng vẫn có thể chặn một số thành phần; shortcut giúp tối ưu thuận lợi hơn nhưng không bảo đảm mọi gradient luôn được giữ nguyên hoặc mạng sâu luôn tốt hơn.

Bảng 4: Các stage của ResNet18 CIFAR; mỗi stage có hai BasicBlock.

Phần	Số kênh	Stride block đầu	Đầu ra
Stem: Conv $3 \times 3 + \text{BN} + \text{ReLU}$	32	1	$32 \times 32 \times 32$
Stage 1	32	1	$32 \times 32 \times 32$
Stage 2	64	2	$64 \times 16 \times 16$
Stage 3	128	2	$128 \times 8 \times 8$
Stage 4	256	2	$256 \times 4 \times 4$
Global average pooling; linear	256	–	$256 \rightarrow 10$

Điều chỉnh cho ảnh nhỏ. Stem dùng kernel 3×3 , stride 1, không max-pooling đầu mạng để giữ chi tiết ở độ phân giải 32×32 . Tám BasicBlock có tổng 16 convolution trên nhánh chính; thêm convolution stem và lớp tuyến tính cuối tạo cấu trúc ResNet18 thông dụng (không đếm riêng convolution projection trong cách gọi này). Base width giảm còn 32, nên mạng có 2.797.610 tham số. Global average pooling lấy trung bình trên 4×4 cho mỗi kênh, giảm số tham số classifier. ResNet có đường truyền gradient thuận lợi, nhưng mức accuracy cuối vẫn phụ thuộc cách chuẩn hóa, điều chuẩn, lịch learning rate và khả năng tổng quát hóa trên dữ liệu đã chọn.

3.5 Vision Transformer: ViT tiny

Bước 1: biến ảnh thành token. Cài đặt dùng convolution kernel 4, stride 4 để nhúng patch. Ảnh $3 \times 32 \times 32$ tạo lưới 8×8 patch; mỗi patch có $4 \times 4 \times 3 = 48$ giá trị và được ánh xạ thành vector 192 chiều. Flatten lưới và chuyển trục tạo tensor 64×192 . Thêm class token và embedding vị trí học được:

$$Z_0 = [x_{cls}; x_p^1 E; \dots; x_p^{64} E] + E_{pos}, \quad E \in \mathbb{R}^{48 \times 192}, \quad E_{pos} \in \mathbb{R}^{65 \times 192}. \quad (7)$$

Phép chiếu patch thực tế còn có bias. Class token là một vector học được, không phải ảnh hay nhãn thật; nó thu thập thông tin cho quyết định phân lớp. Embedding vị trí phân biệt các vị trí trong lưới, vì attention đơn thuần không mã hóa thứ tự không gian [4].

Bước 2: self-attention ba head. Trong mỗi block, LayerNorm chuẩn hóa 192 chiều của từng token; một lớp tuyến tính tạo Q, K, V . Mỗi head có $Q_h, K_h, V_h \in \mathbb{R}^{65 \times 64}$:

$$A_h = \text{softmax}_{\text{hàng}} \left(\frac{Q_h K_h^\top}{\sqrt{64}} \right), \quad U_h = A_h V_h. \quad (8)$$

Mỗi hàng của $A_h \in \mathbb{R}^{65 \times 65}$ biểu diễn trọng số kết hợp các token cho một token truy vấn. Ba U_h được nối thành 192 chiều rồi chiếu bằng lớp tuyến tính. Các head có thể học những kiểu quan hệ khác nhau, nhưng không có ràng buộc rằng mỗi head phải tương ứng một bộ phận vật thể cụ thể. Việc trao đổi thông tin diễn ra giữa mọi cặp token ngay trong một block, khác với tích chập chỉ kết hợp lân cận tại từng lớp.

Bước 3: MLP và kết nối phần dư. Sáu block dùng cấu trúc pre-norm, đúng theo thứ tự trong mã nguồn:

$$Z'_\ell = Z_{\ell-1} + \text{MSA}(\text{LN}(Z_{\ell-1})), \quad (9)$$

$$Z_\ell = Z'_\ell + \text{MLP}(\text{LN}(Z'_\ell)), \quad (10)$$

$$\text{MLP}(u) = W_2 \text{GELU}(W_1 u + b_1) + b_2. \quad (11)$$

MLP mở rộng $192 \rightarrow 768$ rồi thu về 192, xử lý từng token với cùng trọng số; self-attention đảm nhiệm trao đổi giữa token. LayerNorm không dùng thống kê batch hay running statistics. Shape 65×192 không đổi qua sáu block; hai nhánh cộng phần dư giúp giữ thông tin đầu vào block.

Bảng 5: Luồng tensor của ViT tiny trong cài đặt.

Bước	Cấu hình	Đầu ra
Patch embedding	Conv kernel 4, stride 4, 192 kênh	$192 \times 8 \times 8$
Chuỗi token	Flatten lưới; thêm class token và vị trí	65×192
Encoder	6 block; 3 head; MLP ratio 4	65×192
Đầu phân lớp	LayerNorm; lấy class token; Linear	$192 \rightarrow 10$

Bước 4: phân lớp và giới hạn. Sau encoder, LayerNorm được áp dụng cho biểu diễn class token trước lớp tuyến tính 10 logit. Mạng có 2.693.578 tham số, không có dropout trong các block và không dùng pretrained. Chi phí tạo attention tăng theo T^2 với $T = 65$; mỗi head có 4.225 trọng số attention cho một ảnh. Patch lớn làm chuỗi ngắn hơn nhưng gom nhiều pixel vào một token; patch nhỏ tăng số token và chi phí. Trong cấu hình này, ViT có khả năng mô hình hóa quan hệ toàn ảnh nhưng phải học từ lượng dữ liệu tương đối hạn chế. Cấu trúc đó không tự bảo đảm accuracy cao hơn CNN.

3.6 Các phương pháp truyền thống làm mốc so sánh

Pixel được trải phẳng thành 3.072 chiều, giữ thứ tự các vị trí RGB nhưng không có cơ chế tích chập hay chia sẻ trọng số theo không gian. HOG chia ảnh xám thành cell 4×4 , tính histogram 9 hướng gradient, chuẩn hóa trên block 2×2 chồng lấn và nối thành 1.764 chiều [3]. HOG giữ bố cục gradient ở lưới cố định, nhưng không giữ trực tiếp màu RGB.

SIFT phẳng tìm keypoint trong một octave, gán hướng và mô tả mỗi điểm bằng 128 giá trị; tối đa 64 descriptor được xếp theo độ mạnh, nối và đếm số 0, thành 8.192 chiều [9]. Cài đặt này rút gọn SIFT và không dùng Bag of Visual Words. Khi keypoint thay đổi thứ tự giữa các ảnh, một chiều trong vector không còn chắc chắn mang cùng ý nghĩa; việc phóng ảnh nhỏ lên cũng không tạo thêm chi tiết thị giác thực sự.

SVM tuyến tính học các điểm số $s_k = w_k^\top x + b_k$ bằng hinge loss đa lớp và điều chuẩn L2 [2]. Dù có nhiều chiều, ranh giới trong không gian đặc trưng vẫn tuyến tính. Random Forest dùng nhiều cây học từ mẫu bootstrap, chọn ngẫu nhiên đặc trưng khi chia nút và bỏ phiếu nhân [1]. Các cây tạo ranh giới phi tuyến nhưng không có cơ chế chuyên biệt để ghép mẫu cục bộ thành biểu diễn ảnh nhiều tầng. Sáu kết quả của hai bộ phân lớp trên ba loại đầu vào được giữ nguyên trong bảng so sánh; chúng là baseline cho cấu hình học sâu đã khảo sát.

4 Thiết lập thí nghiệm

Bốn mạng được định nghĩa bằng các lớp cơ bản của PyTorch, khởi tạo ngẫu nhiên và huấn luyện độc lập trên MPS; không dùng model zoo hay pretrained weights. Cùng seed 42, số mẫu train/validation/test là 45.000/5.000/10.000. Mỗi mạng chạy 200 epoch với AdamW, learning rate 3×10^{-4} , weight decay 10^{-4} và batch size 128. Gradient norm được giới hạn ở 1,0. Checkpoint có accuracy validation cao nhất được dùng để đánh giá test; checkpoint epoch 200 không mặc nhiên là checkpoint tốt nhất.

Các baseline SVM chạy 200 epoch, batch size 256, learning rate ban đầu 0,01 và điều chuẩn 10^{-4} . Random Forest dùng 200 cây, độ sâu tối đa 15, tối thiểu 4 mẫu để chia nút và 2 mẫu ở nút lá. Mỗi nút xét 16 ngưỡng trên $\sqrt{d}$ chiều đặc trưng chọn ngẫu nhiên. Số epoch và số cây là hai loại ngân sách khác nhau; phép so sánh dùng cùng cỡ dữ liệu và seed, không bảo đảm cùng chi phí tính toán hay mức độ tối ưu siêu tham số.

Với ma trận nhầm lẫn C_{ij} (hàng là nhãn thật, cột là nhãn dự đoán), các chỉ số là

$$\text{Accuracy} = \frac{\sum_k C_{kk}}{\sum_{i,j} C_{ij}}, \quad P_k = \frac{C_{kk}}{\sum_i C_{ik}}, \quad R_k = \frac{C_{kk}}{\sum_j C_{kj}}, \quad F1_k = \frac{2P_k R_k}{P_k + R_k}. \quad (12)$$

Macro-F1 là trung bình F1 của 10 lớp; giá trị bằng 0 được dùng khi mẫu số bằng 0. Các chỉ số trong bảng được trình bày trên thang $[0, 1]$.

5 Kết quả và thảo luận

5.1 Bảng so sánh tổng hợp

Bảng 6: So sánh phương pháp truyền thống và học sâu trên 10.000 ảnh test CIFAR-10.

Nhóm	Cấu hình	N_{test}	Accuracy	Macro-F1
Truyền thống	Pixel-SVM	10000	0.3757	0.3751
Truyền thống	Pixel-RF	10000	0.4509	0.4462
Truyền thống	HOG-SVM	10000	0.5319	0.5305
Truyền thống	HOG-RF	10000	0.4673	0.4593
Truyền thống	SIFT-SVM	10000	0.2292	0.2299
Truyền thống	SIFT-RF	10000	0.1857	0.1433
Học sâu	AlexNet (GroupNorm)	10000	0.8387	0.8381
Học sâu	VGG11-BN	10000	0.7952	0.7944
Học sâu	ResNet18	10000	0.8145	0.8141
Học sâu	ViT tiny	10000	0.6821	0.6803

Bảng 6 tổng hợp sáu cấu hình truyền thống và bốn mạng học sâu. Các baseline lấy từ `outputs/metrics/*_200.json`; kết quả học sâu lấy từ `outputs/deep_models_200_gn/*_metrics.json`.

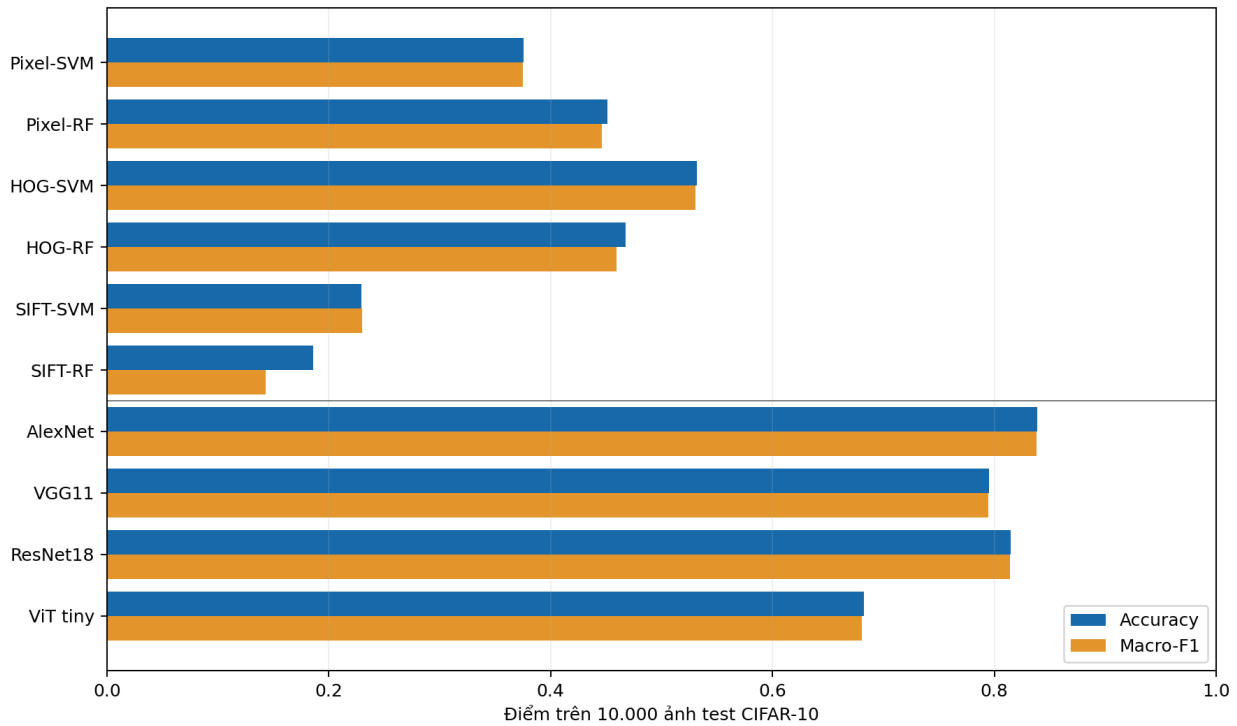
AlexNet đạt accuracy 0,8387 và macro-F1 0,8381, cao nhất trong các cấu hình này. So với baseline tốt nhất HOG-SVM (0,5319 và 0,5305), AlexNet tăng 30,68 điểm phần trăm accuracy và khoảng 30,76 điểm phần trăm macro-F1. ResNet18 đạt 0,8145, VGG11-BN đạt 0,7952 và ViT tiny đạt 0,6821 accuracy. Ngay cả ViT tiny cũng vượt HOG-SVM 15,02 điểm phần trăm accuracy.

5.2 Kết quả huấn luyện các mô hình học sâu

Bảng 7: Checkpoint và chi phí huấn luyện các mô hình học sâu; accuracy trong khoảng $[0, 1]$.

Mô hình	Tham số	Epoch tốt	Val acc.	Train (giờ)
AlexNet (GroupNorm)	2,786,890	193	0.8518	3.48
VGG11-BN	2,310,186	130	0.8120	2.19
ResNet18	2,797,610	184	0.8262	6.51
ViT tiny	2,693,578	180	0.6888	5.18

Các số liệu tương ứng đúng với cấu hình và tập mẫu được ghi ở đây.



Hình 2: Accuracy và macro-F1 trên cùng 10.000 ảnh test. SVM và bốn mạng dùng 200 epoch; Random Forest dùng 200 cây.

Các kết quả có cùng cỡ tập dữ liệu và seed 42, nhưng thuật toán tối ưu và chi phí tính toán khác nhau.

5.3 Đọc bảng so sánh: mức cải thiện và số lỗi

Thứ hạng và ý nghĩa của hai chỉ số. Trong Bảng 6, AlexNet đứng đầu ở cả accuracy và macro-F1; sau đó là ResNet18, VGG11-BN và ViT tiny. Cả bốn mạng đều cao hơn sáu baseline. HOG-SVM là baseline mạnh nhất, đạt 53,19% accuracy, nên được chọn làm mốc phân tích mức cải thiện trong Bảng 8. Đây là mốc đối chiếu kết quả, không phải cấu hình được chọn để điều chỉnh các mạng bằng test.

Bảng 8: Mức cải thiện so với HOG-SVM trên cùng 10.000 ảnh test; “đpt” là điểm phần trăm.

Mạng	Tăng accuracy (đpt)	Tăng macro-F1 (đpt)	Số lỗi	Giảm lỗi tương đối
AlexNet	30.68	30.76	1,613	65.54%
ResNet18	28.26	28.37	1,855	60.37%
VGG11-BN	26.33	26.39	2,048	56.25%
ViT tiny	15.02	14.98	3,179	32.09%

HOG-SVM phân loại sai 4.681 ảnh; AlexNet sai 1.613 ảnh, tức ít hơn 3.068 ảnh. Mức tăng accuracy 30,68 điểm phần trăm tương ứng giảm khoảng 65,54% số lỗi so với baseline, không phải tăng 30,68% tương đối về accuracy. ResNet18, VGG11-BN và ViT tiny lần lượt giảm 2.826, 2.633 và 1.502 lỗi. Vì cùng dùng tập test 10.000 ảnh, số ảnh đúng và sai có thể đối chiếu trực tiếp. Tuy nhiên, số liệu tổng hợp không cho biết chính xác ảnh nào được sửa đúng hay ảnh nào mới bị sai; cần dự đoán ghép theo cùng test index để phân tích đó.

Accuracy và macro-F1 của mỗi mạng khá gần nhau: chênh khoảng 0,06 điểm phần trăm ở AlexNet, 0,04 ở ResNet18, 0,08 ở VGG11-BN và 0,18 ở ViT tiny. Cải thiện vì vậy vẫn thể hiện khi mỗi lớp được cho trọng số ngang nhau trong macro-F1. Điều này không có nghĩa mọi lớp có chất lượng giống nhau; chẳng hạn AlexNet vẫn có F1 của *cat* thấp hơn rõ rệt so với *automobile*. SIFT-Random Forest có accuracy 18,57% nhưng macro-F1 chỉ 14,33%, gợi ý hiệu quả phân lớp mất cân đối hơn và cần kiểm tra chỉ số từng lớp.

5.4 Vì sao học sâu đạt kết quả tốt hơn các baseline?

1. Đặc trưng được học theo mục tiêu phân lớp. HOG và SIFT cố định cách lấy gradient hoặc descriptor trước khi học classifier. Nếu thông tin hữu ích bị bỏ ở bước biểu diễn, SVM hay Random Forest không thể khôi phục từ vector đã mất thông tin. Với mạng sâu, sai số từ nhãn truyền ngược tới các lớp đầu, điều chỉnh cả bộ lọc lẫn bộ phân lớp. Mạng có thể chọn những tổ hợp cạnh, màu và kết cấu phù hợp với 10 lớp trong train, thay vì dùng cùng một quy tắc trích xuất thủ công cho mọi đối tượng. Trong bảng, ưu thế xuất hiện ở cả ba CNN lẫn ViT, phù hợp với lợi ích của học biểu diễn theo nhiệm vụ.

2. Khai thác không gian và thông tin màu. Các mạng nhận ảnh RGB và xử lý cấu trúc ảnh thông qua tích chập hoặc embedding vị trí patch. HOG và SIFT ở đây dùng ảnh xám, nên không trực tiếp giữ khác biệt giữa các mảng màu có mức sáng tương tự. Pixel thô vẫn giữ RGB và thứ tự vị trí, nhưng Linear SVM chỉ áp dụng một tổ hợp tuyến tính trên toàn vector. Khi vị trí vật thể thay đổi, một bộ trọng số gắn với từng tọa độ cố định có thể kém linh hoạt. CNN tái sử dụng cùng bộ lọc ở nhiều vị trí; pooling và các tầng sau kết hợp đáp ứng cục bộ. ViT giữ vị trí token và dùng attention để kết hợp các vùng. Ưu thế này không đồng nghĩa mạng bất biến hoàn toàn với mọi phép dịch chuyển, xoay hay thay đổi nền.

3. Tạo tổ hợp phi tuyến nhiều tầng. SVM tuyến tính không tự tạo các tương tác phi tuyến mới giữa chiều đặc trưng. Random Forest có thể học quy tắc phi tuyến, nhưng các phép chia một chiều không cung cấp cấu trúc học mẫu không gian như convolution hay attention. Ở mạng sâu, convolution/linear, ReLU hoặc GELU và các tầng liên tiếp tạo nhiều mức tổ hợp. Chẳng hạn, một đường viền đơn lẻ có thể xuất hiện ở nhiều lớp; kết hợp đường viền với màu, vị trí và các vùng lân cận có thể giúp phân biệt tốt hơn. Điều này giải thích về mặt cơ chế vì sao tăng độ linh hoạt của biểu diễn có thể nâng accuracy.

4. Tránh vấn đề thứ tự descriptor của SIFT phẳng. SIFT-SVM đạt 22,92% và SIFT-Random Forest đạt 18,57%, thấp hơn cả các baseline pixel. Trong vector SIFT nổi trực tiếp, descriptor thứ nhất của một ảnh có thể nằm ở cánh máy bay, nhưng descriptor thứ nhất của ảnh khác nằm ở nền. Việc đệm 0 còn tạo khác biệt theo số keypoint. Trái lại, CNN giữ lưới không gian qua các feature map; ViT gán embedding vị trí và có phép kết hợp token học được. Cả hai không yêu cầu các điểm mạnh nhất phải xuất hiện theo cùng thứ tự. Kết quả thấp này là hạn chế của SIFT phẳng rút gọn đang dùng, không phải bằng chứng rằng mọi pipeline SIFT đều kém; SIFT-BoVW hoặc spatial pooling có thể thay đổi biểu diễn và cần được đánh giá riêng.

5. Số tham số, tối ưu và điều chuẩn cũng khác nhau. Các mạng có khoảng 2,31–2,80 triệu tham số và được tối ưu bằng AdamW, trong khi SVM là mô hình tuyến tính và Random Forest dùng cấu trúc cây. Chuẩn hóa activation, weight decay, gradient clipping và dropout trong AlexNet góp phần định hình quá trình học. Do đó, bằng chứng minh hiệu quả của toàn bộ cấu hình học sâu đã chạy; chưa tách riêng ảnh hưởng của học đặc trưng, giữ RGB, kích thước mô hình hay optimizer. Muốn xác định nguyên nhân định lượng, cần ablation như CNN trên ảnh xám, classifier tuyến tính trên embedding CNN cố định, hoặc thay đổi một thành phần điều chuẩn trong khi giữ các yếu tố khác giống nhau.

5.5 Vì sao thứ hạng giữa các mô hình lại như trong bảng?

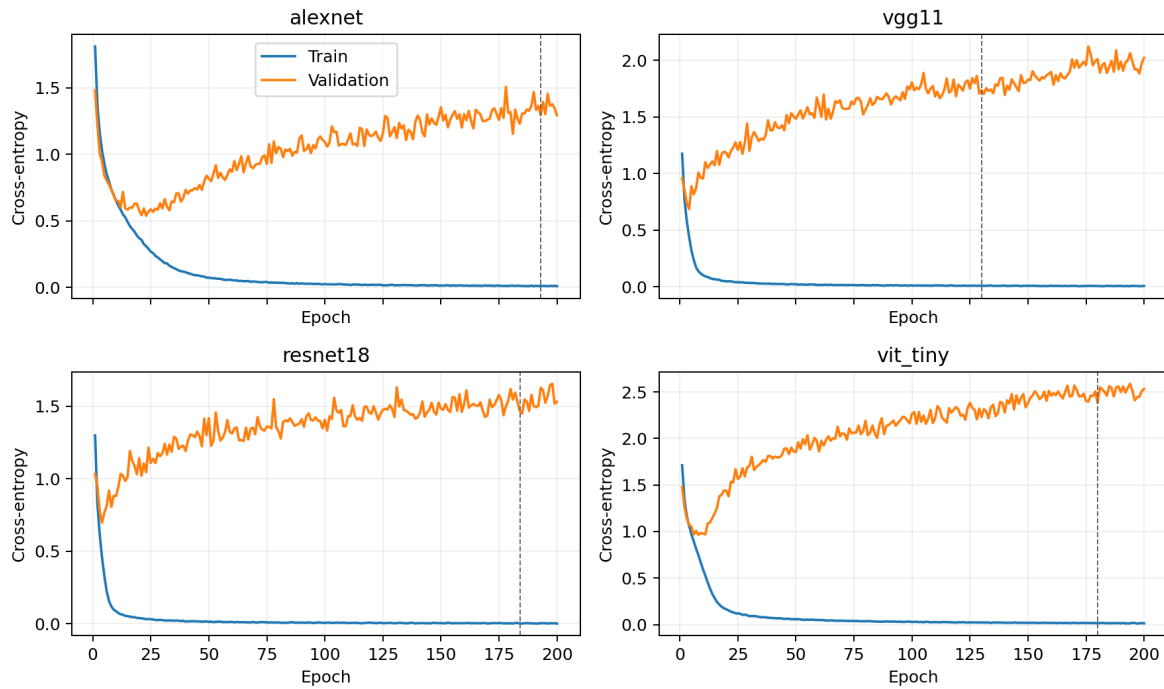
Trong nhóm baseline. Pixel-Random Forest đạt 45,09%, hơn Pixel-SVM 7,52 điểm phần trăm, phù hợp với lợi ích của ranh giới phi tuyến trên đầu vào màu thô. Khi chuyển sang HOG, SVM đạt 53,19%, cao hơn Random Forest 6,46 điểm phần trăm. Đặc trưng histogram nhiều chiều đã mã hóa hình dạng ở lưới cố định, có thể phù hợp hơn với bộ phân lớp dùng đồng thời nhiều chiều. Random Forest giới hạn độ sâu 15 và xét một tập chiều nhỏ ở mỗi nút; việc mô hình hóa các tổ hợp rộng của gradient có thể đòi hỏi nhiều phép chia. Các lý giải này cần kiểm tra bằng điều chỉnh siêu tham số, không đủ để khẳng định SVM luôn tốt hơn Random Forest trên HOG.

AlexNet và hai CNN còn lại. AlexNet cao hơn ResNet18 2,42 điểm phần trăm và VGG11-BN 4,35 điểm phần trăm accuracy. AlexNet có GroupNorm, dropout 0,5 ở classifier và còn giữ lưới 2×2 sau adaptive pooling; VGG dùng pooling tới 1×1 , ResNet dùng global average pooling. Các lựa chọn này ảnh hưởng cách giữ vị trí, khả năng biểu diễn và điều chuẩn. VGG có ít tham số nhất, còn ResNet18 có nhiều nhất, nhưng số tham số không xếp đúng thứ hạng accuracy: lớn hơn hoặc sâu hơn không bảo đảm tốt hơn. ResNet giúp tối ưu mạng qua shortcut; ưu điểm đó không tự giải quyết được quá khớp hay bảo đảm lịch tối ưu chung là phù hợp nhất cho ResNet. Chưa có ablation hoặc nhiều seed để xác định yếu tố nào quyết định mức chênh.

ViT tiny so với CNN. ViT có số tham số gần AlexNet nhưng accuracy thấp hơn 15,66 điểm phần trăm. CNN có thiên kiến cấu trúc về tính cục bộ và chia sẻ bộ lọc; ViT dùng tương tác toàn chuỗi patch và phải học nhiều quan hệ từ dữ liệu. Nghiên cứu ViT chỉ ra vai trò của lượng dữ liệu và thiên kiến cấu trúc khi huấn luyện [4]. Trong thí nghiệm này chỉ có 45.000 ảnh train, không augmentation, không pretrained; ViT cũng không có dropout trong encoder. Các điều kiện này phù hợp với giả thuyết rằng ViT khó tổng quát hóa hơn ba CNN, dù loss train giảm rất thấp. Attention cho phép nhìn toàn ảnh nhưng không bảo đảm sử dụng đúng dấu hiệu vật thể; nền vẫn có thể trở thành tín hiệu gây nhiễu. Kết quả 68,21% vẫn cao hơn HOG-SVM, nên ViT học được biểu diễn hữu ích trong cấu hình này.

Hiệu quả và giới hạn phép so sánh. VGG11-BN mất khoảng 2,19 giờ, ngắn hơn AlexNet 1,29 giờ, nhưng giảm 4,35 điểm phần trăm accuracy. ResNet18 mất khoảng 6,51 giờ, dài hơn AlexNet khoảng 3,03 giờ, trong khi accuracy thấp hơn. Vì vậy, AlexNet có sự cân bằng thuận lợi giữa accuracy và thời gian trong lần chạy này. Thời gian train không thay thế thời gian inference, bộ nhớ cực đại hay chi phí năng lượng. Các mạng dùng MPS; baseline NumPy có cài đặt và thiết bị thực thi khác. Cùng số mẫu và seed giúp đối chiếu dữ liệu, còn 200 epoch và 200 cây không tạo ngân sách tính toán ngang nhau. Một lần chạy chưa đủ để đánh giá độ ổn định hay ý nghĩa thống kê của thứ hạng.

5.6 Diễn biến huấn luyện và chi phí

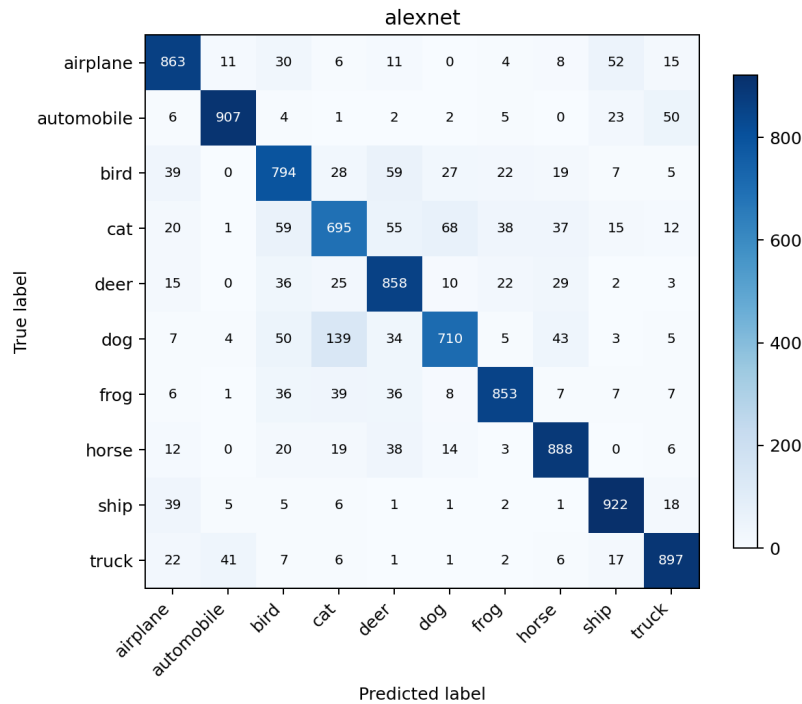


Hình 3: Cross-entropy train và validation của bốn mạng; đường dọc đánh dấu epoch có accuracy validation tốt nhất.

Hình 3 được dựng trực tiếp từ lịch sử 200 epoch trong metrics. Loss train giảm xuống rất thấp, trong khi loss validation tăng về cuối quá trình ở nhiều mạng. Chênh lệch này cho thấy dấu hiệu overfitting; accuracy validation và cross-entropy không nhất thiết biến thiên cùng chiều. Vì vậy, việc chọn checkpoint theo validation thay vì dùng epoch cuối là cần thiết. Các epoch được chọn lần lượt là 193, 130, 184 và 180 cho AlexNet, VGG11-BN, ResNet18 và ViT tiny.

Bảng 7 cho thấy VGG11-BN mất khoảng 2,19 giờ huấn luyện, AlexNet 3,48 giờ, ViT tiny 5,18 giờ và ResNet18 6,51 giờ. Trong lần chạy này, VGG11-BN có thời gian ngắn nhất, còn AlexNet cho accuracy cao nhất. Đây là thời gian được ghi bởi chương trình trong quá trình train, bao gồm đánh giá validation và lưu checkpoint, không phải benchmark suy luận độc lập. Chi phí còn phụ thuộc thiết bị và cài đặt; 200 epoch giữa các kiến trúc không đồng nghĩa với cùng lượng tính toán.

5.7 Phân tích lỗi của mô hình tốt nhất



Hình 4: Ma trận nhầm lẫn của checkpoint AlexNet được chọn theo validation; hàng là nhãn thật, cột là nhãn dự đoán, mỗi lớp có 1.000 ảnh test.

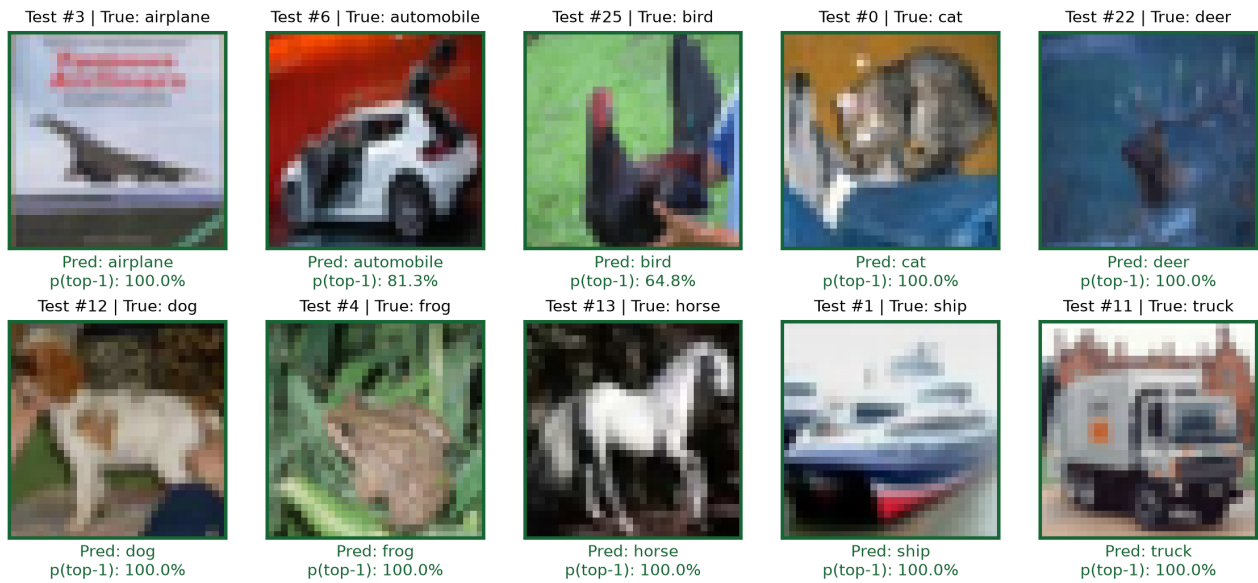
Các giá trị trong Hình 4 lấy từ kết quả test đã lưu. AlexNet đạt F1 cao nhất ở *automobile* (0,9208) và *ship* (0,9004), trong khi *cat* có F1 thấp nhất (0,7077) và recall 0,695. Cặp nhầm lẫn lớn nhất là *dog*→*cat* với 139 ảnh; chiều ngược lại có 68 ảnh. Hai cặp *bird*→*deer* và *cat*→*bird* đều có 59 ảnh. Như vậy, học sâu cải thiện kết quả tổng thể nhưng vẫn gặp khó khăn ở một số lớp động vật. Hình dáng gần nhau ở độ phân giải thấp là một cách giải thích cần được kiểm tra thêm bằng các ảnh dự đoán sai.

5.8 Minh họa kết quả dự đoán trên cùng ảnh test

Các hình dưới đây được sinh bằng suy luận từ checkpoint tốt nhất đã lưu, không huấn luyện lại. Ảnh giữ nguyên RGB 32×32 , chỉ được phóng lớn bằng nearest-neighbor để nhìn rõ; không dùng ảnh tổng hợp. Mỗi mạng dùng đúng mean và std lưu trong checkpoint, chế độ `eval()` và `inference_mode()`, suy luận trên CPU cho phần minh họa. Cùng 10 ảnh được dùng cho cả bốn mạng: lấy ảnh đầu tiên của từng lớp theo thứ tự file test chính thức, tương ứng index 3, 6, 25, 0, 22, 12, 4, 13, 1 và 11. Quy tắc chọn ảnh không phụ thuộc kết quả dự đoán.

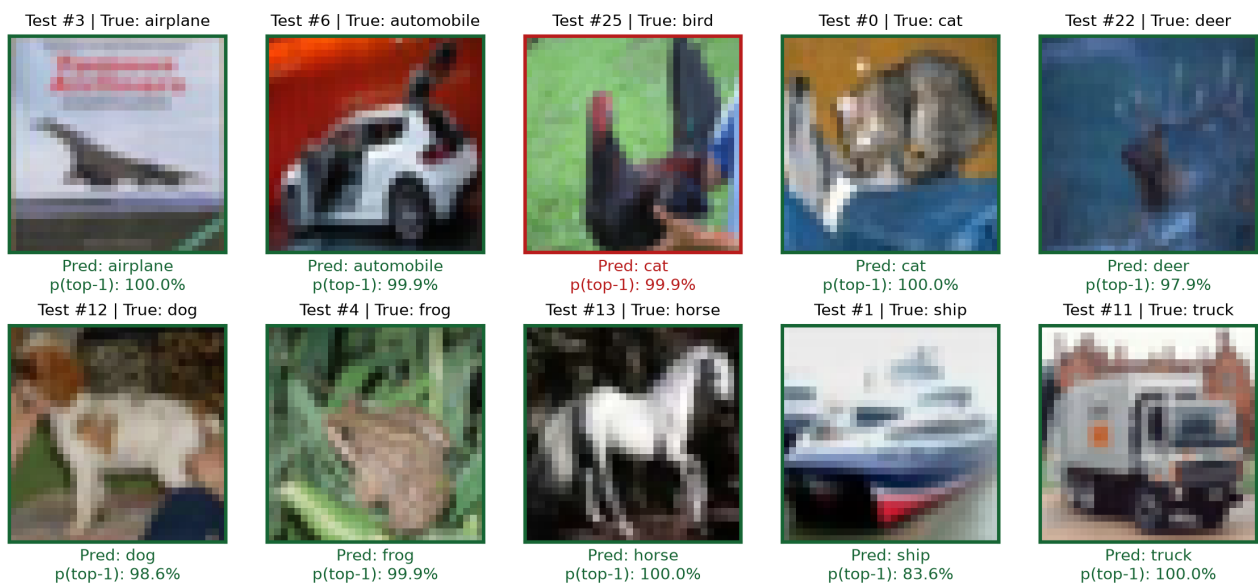
Trên hình, True là nhãn thật, Pred là nhãn dự đoán; viền xanh biểu thị đúng, viền đỏ biểu thị sai. $p(\text{top-1})$ là giá trị softmax của lớp có điểm cao nhất, làm tròn đến một chữ số thập phân theo phần trăm; số hiển thị 100,0% có thể là giá trị gần 1 đã được làm tròn. Đây là điểm mô hình, không phải accuracy của ảnh hay xác suất đã được hiệu chuẩn. Mười ảnh minh họa không thay thế đánh giá trên đủ 10.000 ảnh test trong Bảng 6.

AlexNet (GroupNorm)



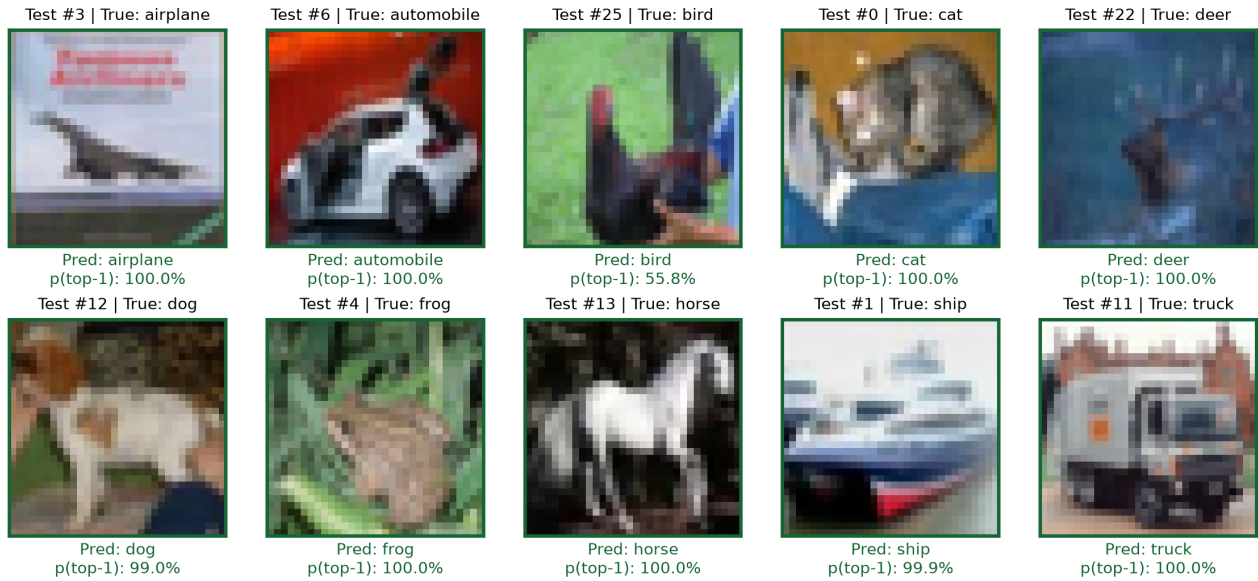
Hình 5: AlexNet: checkpoint epoch 193, cùng 10 ảnh test, mỗi lớp một ảnh.

VGG11-BN



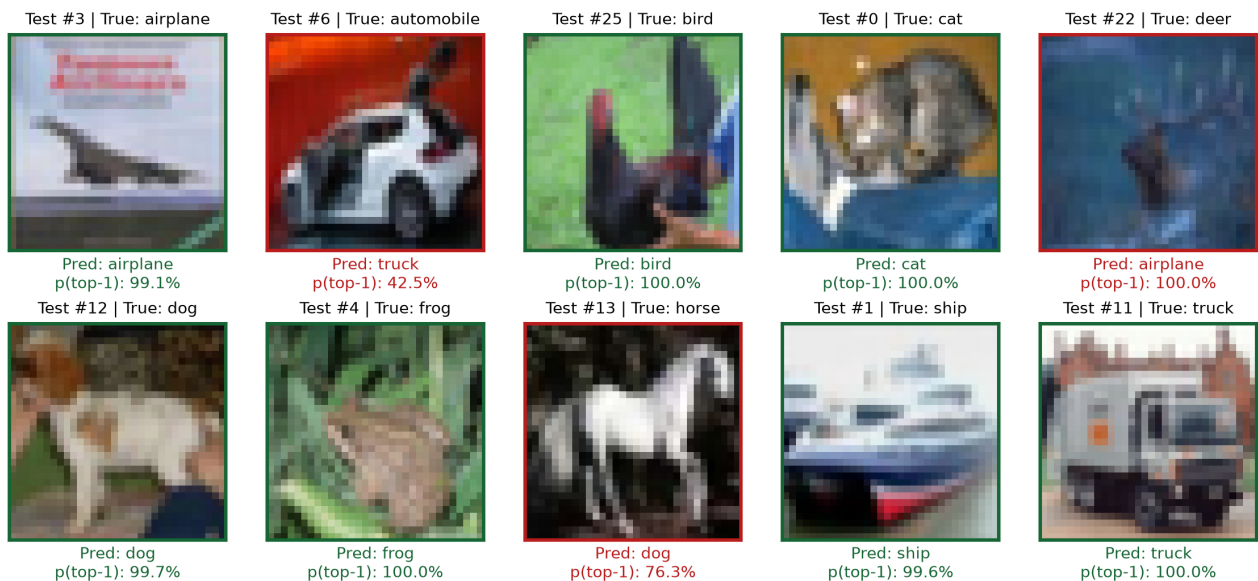
Hình 6: VGG11-BN: checkpoint epoch 130, các ảnh và thứ tự giống Hình 5.

ResNet18



Hình 7: ResNet18: checkpoint epoch 184, các ảnh và thứ tự giống Hình 5.

ViT tiny

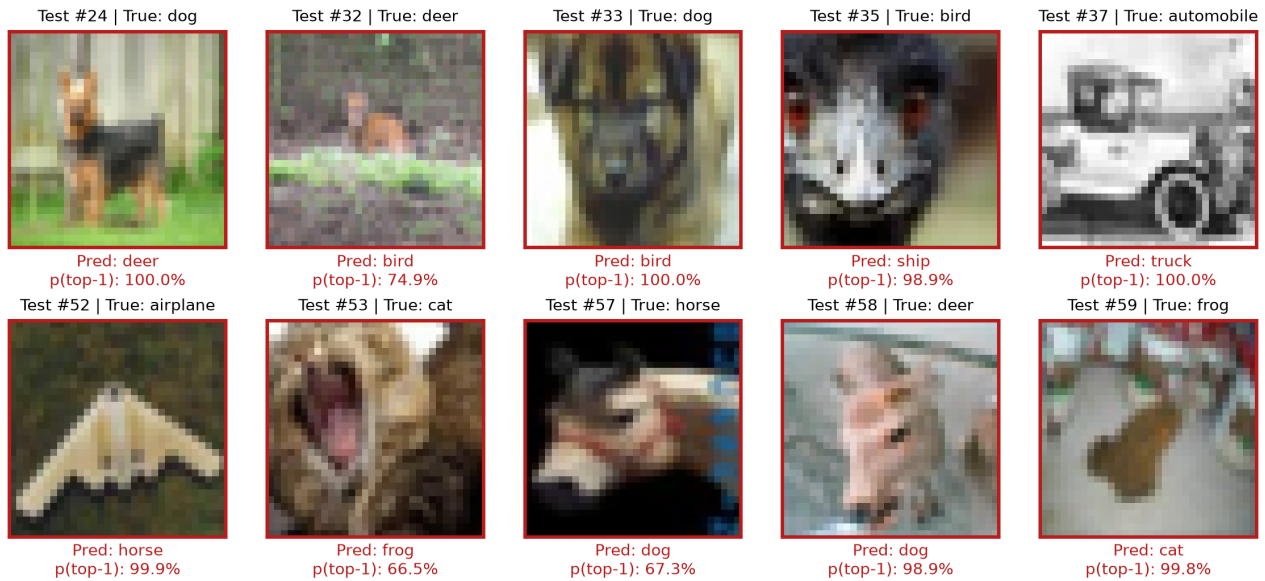


Hình 8: ViT tiny: checkpoint epoch 180, các ảnh và thứ tự giống Hình 5.

Đôi chiếu cùng mẫu. Ảnh #25 có nhãn *bird*: AlexNet và ResNet18 phân loại đúng với softmax top-1 khoảng 64,8% và 55,8%, trong khi VGG11-BN nhầm thành *cat* với 99,9%. ViT tiny phân loại đúng ảnh này, dù accuracy tổng thể thấp hơn VGG11-BN. Điều đó cho thấy mô hình đúng cao hơn trong bảng không nhất thiết đúng trên mọi ảnh mà mô hình thấp hơn làm đúng. Ở ảnh #6, ViT nhầm *automobile* thành *truck* với 42,5%; ba CNN đều đúng. ViT còn nhầm #22 từ *deer* thành *airplane* và #13 từ *horse* thành *dog*. Các sai khác minh họa tính phụ thuộc mẫu của quyết định và giới hạn khi diễn giải một nhóm ảnh nhỏ.

5.9 Ví dụ dự đoán sai và hiện tượng tự tin quá mức

AlexNet: first 10 errors in test #0-499



Hình 9: Mười lỗi đầu tiên của AlexNet trong 500 ảnh test đầu (index 0–499), sắp theo index tăng dần; nhóm này được chọn có điều kiện là dự đoán sai.

Hình 9 được chọn theo một quy tắc khác với nhóm mỗi lớp một ảnh: lấy mười lỗi đầu tiên trong 500 ứng viên để quan sát kiểu lỗi, không để ước lượng accuracy. Các index là 24, 32, 33, 35, 37, 52, 53, 57, 58 và 59. Ảnh #37, *automobile*→*truck*, có hai lớp cùng đặc điểm xe và đường nét gần nhau. Ảnh #57, *horse*→*dog*, cho thấy khó khăn ở động vật có nhiều phần thân không rõ tại độ phân giải thấp. Ảnh #52, *airplane*→*horse*, có hình dáng vật thể và nền khó tách khi phóng lớn. Những quan sát này gợi ý dấu hiệu thị giác có thể gây nhầm; ảnh minh họa chưa xác định chính xác vùng nào khiến mạng ra quyết định.

Một số lỗi có softmax rất cao: #24 nhầm *dog* thành *deer* với khoảng 99,9995%, #37 nhầm xe với khoảng 99,9614%. Do đó không nên dùng riêng ngưỡng softmax cao để xem một dự đoán là chắc chắn đúng. Hiện tượng này cũng phù hợp với đường validation loss tăng dù train loss gần 0: cross-entropy phạt mạnh dự đoán sai có độ tự tin cao. Để đo mức hiệu chuẩn hoặc kiểm tra nguyên nhân lỗi, cần đánh giá riêng các chỉ số hiệu chuẩn và phân tích vùng ảnh; báo cáo hiện chỉ kết luận trực tiếp về nhân và điểm đã suy luận.

Thông tin nguồn checkpoint, epoch, SHA-256, index ảnh, nhãn và điểm dự đoán được lưu tại `report/assets/results/prediction_examples.json`. Các phân bố softmax của 500 ảnh ứng viên cho mỗi mạng được lưu trong `report/assets/results/*_predict_examples.npz`, phục vụ tái kiểm tra.

6 Kết luận

Báo cáo đã cài đặt và đánh giá các biến thể AlexNet, VGG11-BN, ResNet18 và ViT tiny trên CIFAR-10, đồng thời giữ kết quả Pixel, HOG và SIFT phẳng kết hợp SVM hoặc Random Forest để so sánh. Trên 10.000 ảnh test chính thức, AlexNet đạt 83,87% accuracy và macro-F1 0,8381, vượt baseline tốt nhất HOG-SVM 30,68 điểm phần trăm accuracy. ResNet18, VGG11-BN và ViT tiny cũng cao hơn toàn bộ baseline trong bảng.

Kết quả cho thấy lợi ích của học biểu diễn trực tiếp từ ảnh trong cấu hình đã khảo sát. CNN đạt kết quả tốt hơn ViT tiny khi cùng huấn luyện từ đầu, không dùng tăng cường dữ liệu; tuy nhiên một seed và một bộ siêu tham số chưa đủ để xếp hạng tổng quát các kiến trúc. Hướng cải thiện tiếp theo là tăng cường dữ liệu, điều chỉnh lịch learning rate và điều chuẩn bằng validation, sau đó lặp lại với nhiều seed. Tập test cần tiếp tục được giữ cho đánh giá cuối.

Tài liệu tham khảo

- [1] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.
- [2] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.
- [3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2005.
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*, 2015.
- [7] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [8] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [9] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [10] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.
- [11] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision*, 2018.